

Architecture and Activity Reference

From a typed sentence to a running container - every hop, with the technology on each one.

WHAT IT IS

An AI app builder. Someone describes an app; a model writes React and TypeScript; the browser compiles it; the result is published as a static site or shipped as a Docker container with its own domain, TLS and database.

TWO INDEPENDENT SYSTEMS

A Vercel deployment serving the pages and one serverless function, and a VPS running a Docker stack. They share a git repository and nothing else - they talk over HTTP, and neither imports a line of the other's code.

HOW TO READ THIS

Pages 3 to 6 are UML activity diagrams with swimlanes: a column per participant, time running down the page. Dashed arrows cross a trust or machine boundary. Every route name, timeout and image tag is quoted from the source.

THE SHAPE OF IT

131	99	3	16	4	300s	4	3
HTTP routes	under /api	path prefixes reach Express	static pages	effort levels	function ceiling	framework families the plane builds	Docker networks

ACTIVITY DIAGRAM NOTATION



THE ONE THAT CATCHES PEOPLE

In production only /api/*, /auth/* and /s/* reach Express. Everything else is a static file, or a 404 at the edge before the function is invoked. So backend/index.js contains routes that cannot run on souqi.site - a 404 from production means "not routed here", not "refused".

THE OTHER ONE

git push deploys nothing. Vercel is shipped from the CLI and the container plane from ship.sh. A push updates GitHub and leaves both running systems exactly as they were - which is why a deploy is verified against a marker that exists only in the new code.

Deployment architecture

What runs where, and on what.



EXTERNAL SERVICES

MongoDB Atlas

projects, revisions, users, sessions, usage, audit
mongodb 6.8

DeepSeek

every model route; per-route breaker, spend line and budget
OpenAI-compatible HTTP

S3 / R2

uploaded images and deployment source archives
hand-rolled SigV4

Stripe

checkout and the webhook that grants a plan
js.stripe.com/v3

Hetzner Cloud

provisioning a host for the plane
REST, provider interface

WHY API/ CANNOT MOVE

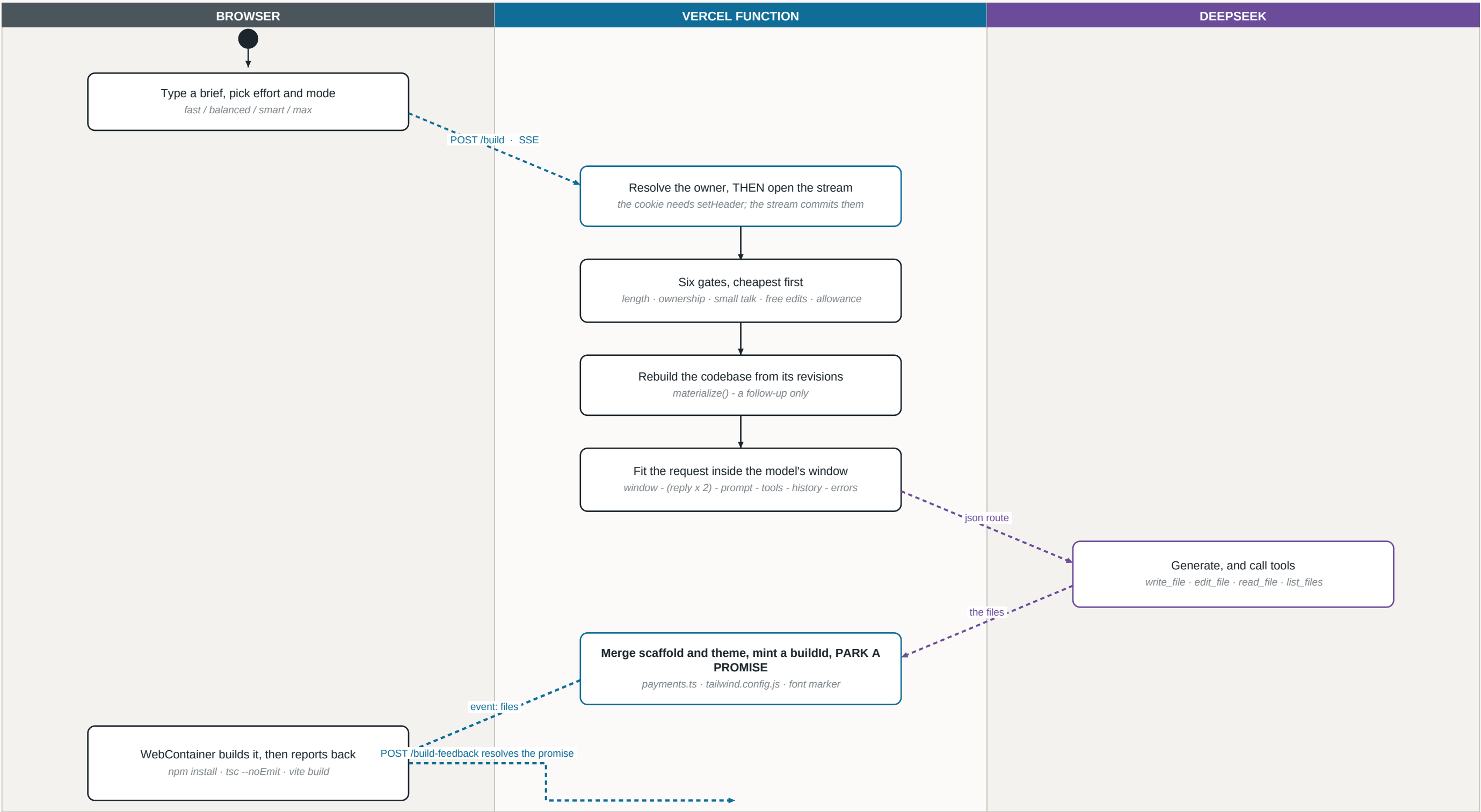
Vercel discovers serverless functions only at `<root>/api/`, and `vercel.json` names `api/index.js` by path. It is a one-line shim so the same Express app runs as a function in production and as a plain server locally - `backend/index.js` exports the app when `process.env.VERCEL` is set and calls `listen()` when it is not.

WHY THE SCAFFOLD SHIPS AS JSON

Included as a directory, Vercel's bundler transpiles the TypeScript it finds - `App.tsx` becomes `App.js` - while `index.html` still asks for `main.tsx`, and the deployed app dies with an empty dist. Excluded without the blob, Rollup cannot resolve `/src/main.tsx`. A `.json` is neither compiled nor dropped, so `scaffold-data.json` is what ships.

Activity - a message becomes a built app

One POST, one long-lived SSE response. The compile happens on the visitor's machine.



continued on page 4 - the repair loop

THE COMPILE IS NOT ON THE SERVER

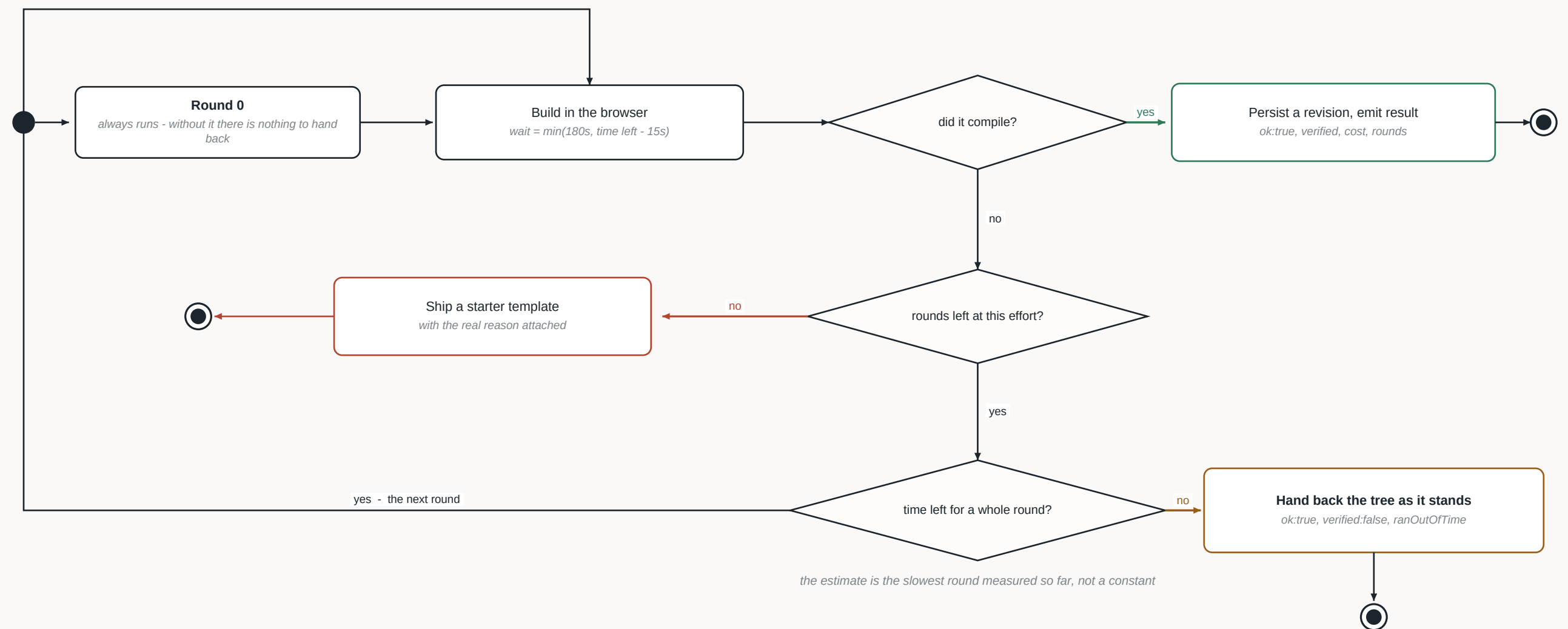
The server writes code, emits it, and blocks on a promise keyed by a random buildId until a completely separate HTTP request resolves it. tsc --noEmit runs because Vite's build is esbuild, which strips TypeScript types without checking them - without it a genuinely broken app compiles clean and the repair loop never learns there was anything to fix.

WHEN THE BROWSER CANNOT BUILD

Every phone, and any browser without SharedArrayBuffer, sends canBuild:false. That skips the client-build loop entirely - one model call, files returned directly, and verified:false on the result, so the audit does not record an unverified build as a success.

Activity - the repair loop and its two budgets

A round is bounded by the effort level AND by the wall clock the platform enforces.



WHAT THIS GUARD IS FOR

vercel.json gives the function maxDuration:300 and the platform does not negotiate. Max effort allows four rounds, each waiting up to three minutes for the browser build, so the process was killed mid-round with the files sitting in memory: no result frame, no error frame, and a client that threw "No result came back" over eleven files it had just watched get written. Twice in a row, five minutes each, for nothing.

THE EFFORT SCALE

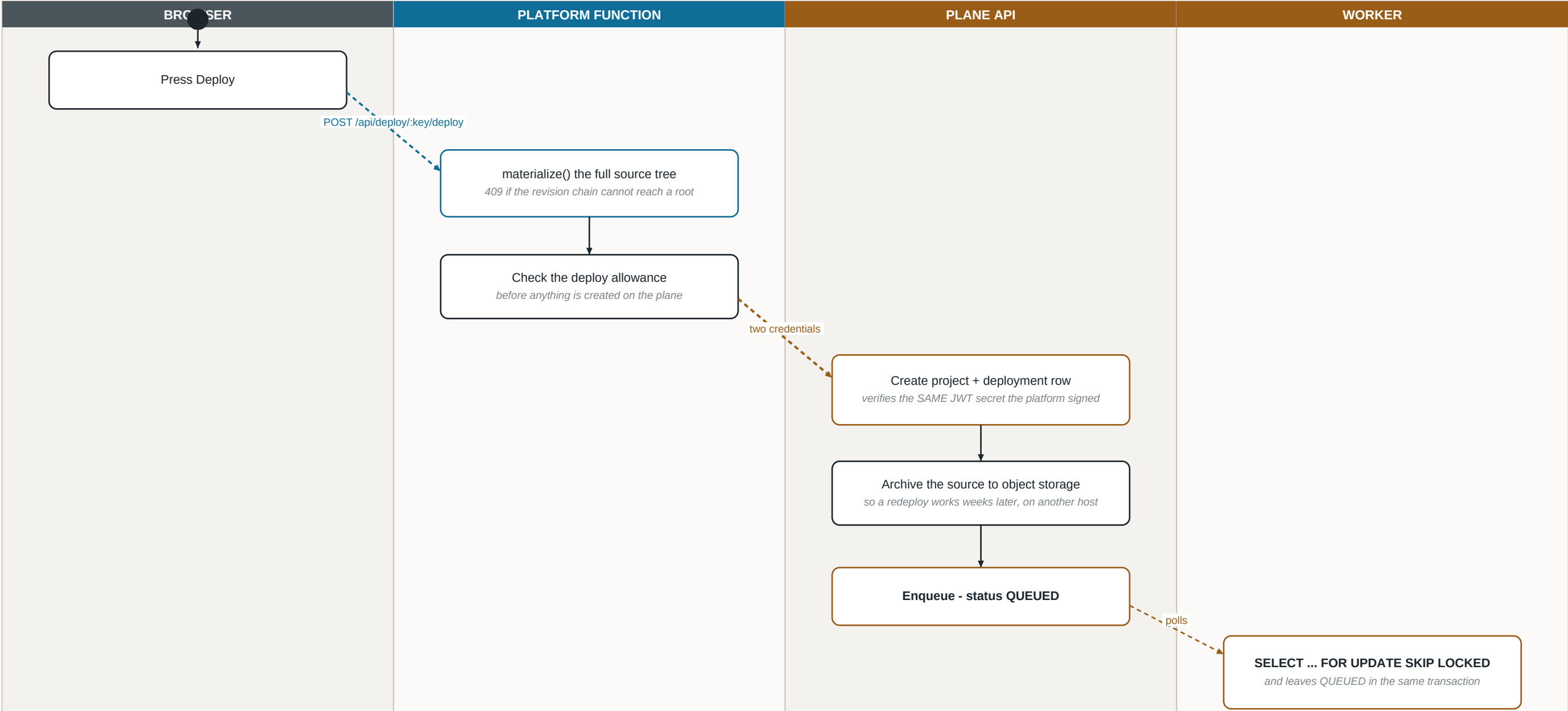
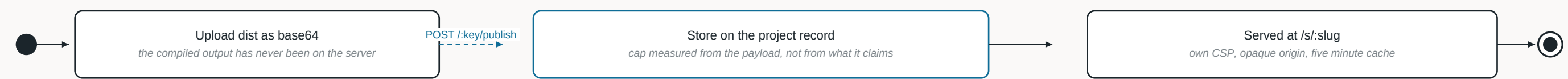
LEVEL	MAX_TOKENS	REPAIR_ROUNDS	MODEL_TIER
fast	16,000	1	eco - deepseek-flash
balanced	32,000	2	eco - the default
smart	48,000	3	power - deepseek-v4-pro
max	64,000	4	power

The reply budget is subtracted twice: once for the reply, once for the attempt being repaired.
Characters convert at 3.0 per token; history caps at 6,000 characters and build errors at 4,000.

Activity - publishing, and handing an app to the plane

Two different outputs: static bytes served from the CDN, or a container with a domain of its own.

PUBLISH - THE STATIC PATH



continued on page 6

TWO CREDENTIALS, TWO QUESTIONS

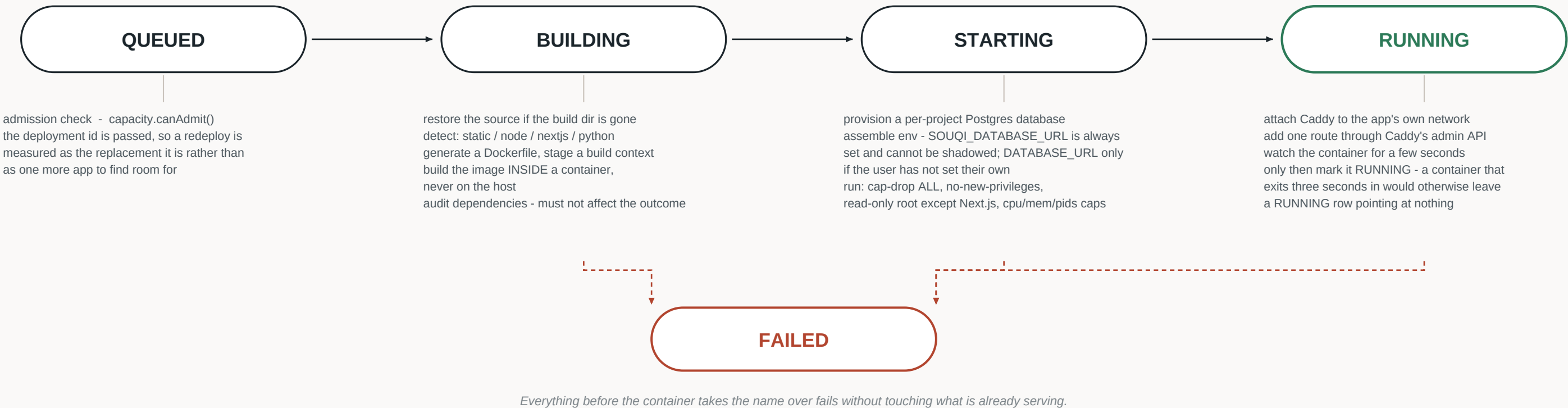
Every call to the plane carries an x-platform-token header - this call came from Souqi - and the caller's own sq_session cookie, forwarded - and this is who is asking. The plane verifies that cookie with the same JWT secret the platform signed it with, so there is one identity across both systems. Both checks apply; neither substitutes for the other.

WHY THE CLAIM IS TRANSACTIONAL

FOR UPDATE SKIP LOCKED is a real work queue: two workers polling the same table never hand out the same row. Moving it out of QUEUED inside the claiming transaction is what stops a crash between claiming and building from leaving a deployment invisible to every worker.

The build pipeline as a state machine

QUEUED to RUNNING, and what each transition actually does.



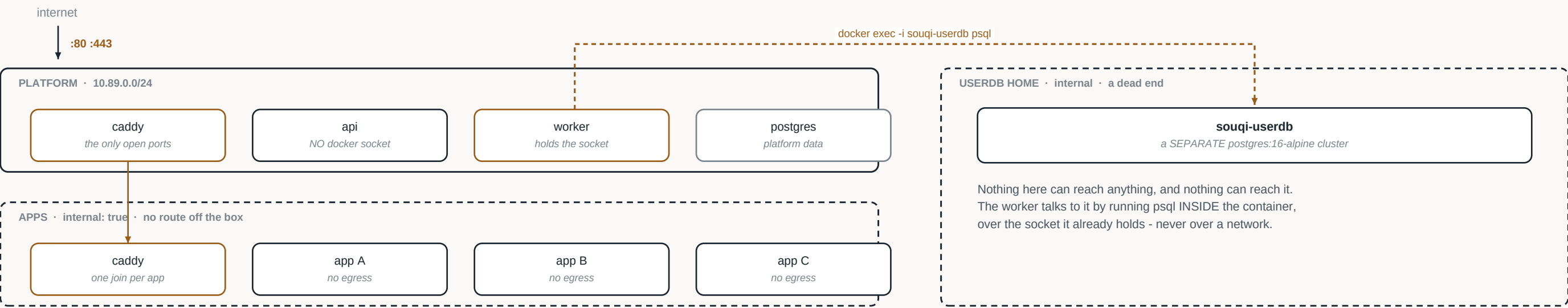
WHAT THE PLANE CAN BUILD			
FRAMEWORK	DETECTED BY	BUILT WITH	SERVED BY
static	vite or react-scripts + a build script	node:20-alpine	nginx:1.27-alpine, unprivileged, :8080
node	any package.json with a start script	node:20-alpine	the app's own server
nextjs	a next dependency	node:20-alpine	next start, writable root for .next
python	requirements.txt / manage.py / main.py	python:3.12-slim	a WSGI or ASGI server

A project can override all of it with a deploy.json declaring framework, buildCommand, startCommand and port.

Isolation, layer by layer

Three mechanisms stacked on each other: the network, the container runtime, and the database.

LAYER 1 · NETWORK · three Docker networks, two of them with no route off the box



LAYER 2 · DOCKER · one process may speak to the daemon, and it never speaks through a shell

The socket is held by exactly one service

The worker mounts `/var/run/docker.sock`. The api does not, and the compose file says so in as many words: the control plane must never be able to run a container. A request that reaches the api can queue work; only the worker can act on it.

Every docker call is an argv array

engine.js builds arguments as a list and hands them to `execFile` - never a command string, never a shell. A project name full of shell metacharacters is an argument, not syntax.

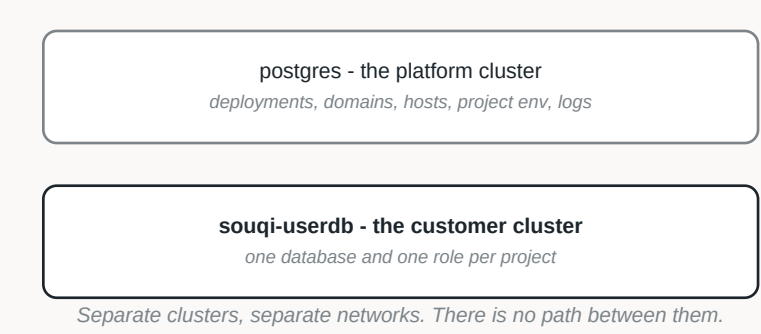
The image is built inside a container

Never on the host. The build context is `scratch`: staged, used and wiped afterwards, which is why a redeploy weeks later restores the source from object storage first.

The runtime flags are not optional extras

`--cap-drop ALL`, `--security-opt no-new-privileges`, a `--read-only root` (except `Next.js`, which caches into `.next`), and `--memory / --cpus / --pids-limit` taken from the deployment row.

LAYER 3 · POSTGRES · two clusters, and a tenant that cannot stand anywhere but its own database



Once, for the whole cluster - `db/userdb-init.sql`

```
REVOKE CONNECT ON DATABASE postgres FROM PUBLIC;
REVOKE CONNECT ON DATABASE template1 FROM PUBLIC;
```

Postgres lets `PUBLIC` connect to the maintenance databases. Left alone a tenant could connect to postgres instead of its own database and work from there - read the catalogs, create objects in a shared schema. Its own database being locked down would not matter, because it never needed to go there. A new database is cloned from `template1`, so revoking there means every future tenant inherits the lockdown.

Per project - `dbproviders/builtin.js`

```
CREATE ROLE <role> LOGIN PASSWORD <...> CONNECTION LIMIT 20
CREATE DATABASE <db> OWNER <role>
REVOKE ALL ON DATABASE <db> FROM PUBLIC
GRANT CONNECT, TEMPORARY ON DATABASE <db> TO <role>
REVOKE ALL ON SCHEMA public FROM PUBLIC · ALTER SCHEMA public OWNER TO <role>
ALTER ROLE <role> SET idle_in_transaction_session_timeout = ...
```

IDENTIFIERS CANNOT BE PARAMETERISED, WHICH IS WHY THIS CODE LOOKS THE WAY IT DOES

`CREATE DATABASE $1` is not a thing - a placeholder can carry a value but never a name. So the provider quotes database and role names explicitly rather than interpolating them, and makes `CREATE ROLE` idempotent with a `DO` block, because there is no `CREATE ROLE IF NOT EXISTS`. The whole exchange runs through `docker exec -i souqi-userdb psql` on the socket the worker already holds; the cluster has no listening port anything could reach.

Technology, by layer

Versions as declared in the manifests.

THE FUNCTION · resolved from the ROOT package.json, because Vercel installs from the repo root

PACKAGE	VERSION	WHAT IT DOES HERE
express	^4.19.2	the whole HTTP surface; 131 routes in one file
mongodb	^6.8.0	projects, revisions, sessions, usage, audit
pg	^8.22.0	per-workspace Postgres for customer app data
jsonwebtoken	^9.0.2	the sq_session cookie, shared with the deploy plane
bcryptjs	^2.4.3	password hashing
@anthropic-ai/sdk	^0.122.0	the bring-your-own-key path; Souqi's own routes are plain HTTP
archiver	^7.0.1	zip export; root-only, the one dependency backend/ does not declare
cross-spawn / cors / dotenv	-	process spawning, CORS, and env loaded from an explicit path

THE BROWSER · no build step; what is in frontend/ is what is served

@webcontainer/api	1.x	the build sandbox, from cdn.jsdelivr.net as an ES module
react / react-dom	18.3.1 UMD	the srcdoc preview fallback where SharedArrayBuffer is absent
babel-standalone	7.24.7	transpiles the fallback preview in the page
Jost + Inter	Google Fonts	display and body; JetBrains Mono is not used by the product

THE PLANE · its own manifest, four dependencies, nothing shared with backend/

express	^4.19.2	the control-plane API
pg	^8.22.0	the platform cluster; the customer cluster is reached by docker exec
jsonwebtoken	^9.0.3	verifies the platform's own session cookie
dotenv	^17.4.2	config, loaded relative to the file not the cwd
postgres:16-alpine	image	two separate clusters: platform data and customer data
caddy:2-alpine	image	TLS and routing, configured through the admin API at runtime
node:20-alpine	image	the plane's own api and worker processes

WHAT THE AGENT WRITES · one fixed stack, so the scaffold, conventions and error signatures are fixed too

react / react-dom	^18.3.1	the generated app
vite	^5.3.1	the build; esbuild under it, which is why tsc runs separately
typescript	^5.5.3	tsc --noEmit is the type check esbuild does not do
tailwindcss	^3.4.4	the config is generated per build from a contrast-checked palette
vite-plugin-pwa	^0.20.5	installability for generated apps

Where data rests, and what crosses a boundary

Four stores on two machines, with one deliberate separation.

STORES			
STORE	HOLDS	LIVES ON	WRITTEN BY
MongoDB Atlas	projects, revisions, users, sessions, usage, audit	external	the platform function
Postgres platform	deployments, domains, hosts, project env, logs	the VPS	plane api + worker
Postgres userdb	each customer app's own data	the VPS, isolated network	worker, via the socket
S3 / R2	uploaded images, deployment source archives	external	both systems
the project record	published dist bytes, base64	MongoDB	the browser, on publish

WHAT NEVER MOVES

Image bytes	Uploaded once and referenced by id on every subsequent turn - the composer sends ids, never bytes.
API keys	A bring-your-own key is stored server-side on the account. The browser sends an intent, not a credential.
Compiled output	Built and previewed entirely in the browser. It reaches the server only when someone presses publish.
Between the two Postgres clusters	There is no path. Not a permission grant - no network, no route, no shared cluster.

VERIFYING A DEPLOY

A route that answers, a page that loads, a 401 where you expected one - none of these distinguish the new deploy from the one before it. Check a marker that exists only in the new code, or compare the served byte count against the local file.

SCHEMA CHANGES ON THE PLANE

db/schema.sql is applied by docker-entypoint-initdb.d, which only runs against a FRESH Postgres volume. An existing host never picks up a schema change from a ship alone - it needs docker compose exec -T api node scripts/migrate.js run explicitly.